

PRÁCTICA DE INFORMÁTICA / EXPRESIÓN GRÁFICA

Aprendizaje basado en proyectos de ingeniería

1. Introducción



En las plantas de procesamiento, una de las herramientas de automatización de procesos más versátiles que existen son los brazos robóticos.

Con una forma en un principio parecida a la de un brazo humano, son capaces de realizar tareas de colocación, apriete, o clasificación, entre otras.

La versatilidad proviene de la posibilidad de manejar diferentes herramientas en su extremo, que comúnmente se llama *manipulador*.

En el caso propuesto, el diseño del robot consiste en dos brazos articulados, y un manipulador en forma de garra que permite que éste pueda coger y soltar objetos.

2. Descripción

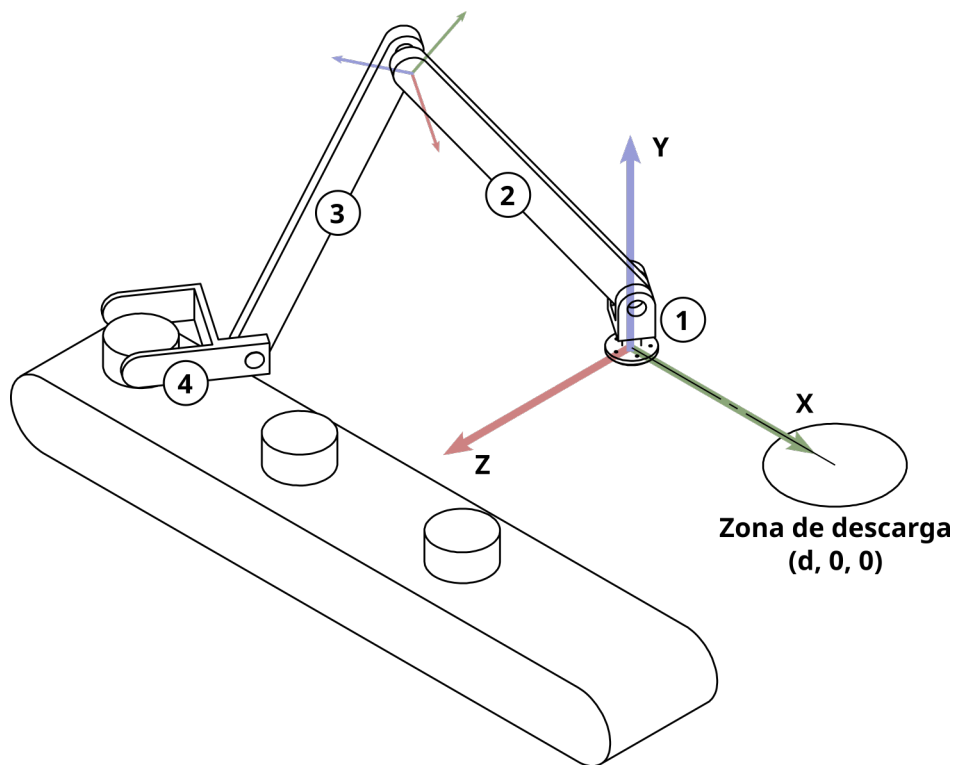
La tarea que se programa en el control del robot es la siguiente: *capturar de una cinta transportadora las latas que viajan por ella, y estibarlas en columnas en otra sección del área de control*, como se muestra en la figura inferior.

El sistema de coordenadas está definido por un eje Y vertical, y el plano ZX horizontal, con su origen en el punto base del robot.

La cinta transportadora tiene una longitud de 1 m, y a lo largo de ella están dispuestas 3 latas, en las coordenadas $L_1 = (-0.4, 0, 0.5)$ m, $L_2 = (0, 0, 0.5)$ m y $L_3 = (0.4, 0, 0.5)$ m.

El robot puede capturarlas en cualquier orden, pero tiene que dejarlas en las coordenadas de la zona de descarga $(0, 0.38, 0)$ m. Si hay ya una lata en la zona de descarga, entonces la tiene que depositar encima de la que ya esté allí.

Las latas tienen unas dimensiones de 100 mm de diámetro por 50 de alto.



3. Determinación de la secuencia de matrices de transformación (Álgebra)

Además del manipulador, el robot está compuesto por tres partes principales:

1. *Base*: permite el giro sobre el eje vertical (Y).
2. *Brazo1*: es el que está directamente conectado a la base, y únicamente puede rotar sobre su articulación con ésta (rotación en el eje X o en el eje Z, dependiendo de la posición de partida).
3. *Brazo2*: el brazo entre el primero y el manipulador. Sólo puede rotar en el eje X / Z, igual que su predecesor. La articulación con el manipulador es libre.
4. *Manipulador*: como la articulación es libre, se mueve con el extremo del brazo2, pero mantiene en todo momento su orientación plana.

4. Creación de la geometría 3D mediante CAD (Expresión Gráfica)

Se pide diseñar las piezas 1, 2 y 3.

Para cada uno de los ficheros SLDPRT, se debe generar también el fichero de plano descriptivo SLDDRW.

Los planos pedidos son de fabricación, por lo que deben indicar todas las medidas necesarias para poder definir la pieza.

La forma de la pieza tiene que estar definida con un nivel de detalle que podría llegar a las 15 operaciones.

Las articulaciones tienen que estar correctamente modeladas.

Guías de modelado: las piezas de máquinas son habitualmente huecas.

El material empleado, acero, tiene una alta característica de resistencia, lo que también ayuda a evitar el sobrepeso de las piezas.

No se pide el modelado de actuadores, motores u otros elementos auxiliares, salvo los pasadores.

Se suministrarán los ficheros correspondientes a las partes recuadradas en blanco en la figura, es decir, todo el resto de la máquina que no sean las 3 piezas pedidas.

Para la modelización de las piezas de la máquina, se deben buscar referencias de máquinas reales como las de la tipología de la figura. Estas referencias visuales (imágenes, vídeos) se deben adjuntar en la entrega final. No es necesario modelizar con total detalle cada uno de los brazos de la grúa, pero se valorará la aproximación al original.

Además de las referencias que se han empleado para la modelización de las piezas (imágenes, enlaces a páginas de fabricantes, vídeos de funcionamiento...), se necesita entregar los ficheros de SolidWorks de cada una de ellas (.SLDPRT) y los planos de cada una de ellas, en el formato de papel (A3, A4) que se considere oportuno (.SLDDRW).

Como de costumbre, además del resultado final en las piezas se busca que se hayan definido todos los croquis 2D empleados. En los planos, que se emplee el número adecuado de vistas, se cumplan las normas de representación y se indiquen todas las medidas necesarias para la fabricación del elemento representado.

Es conveniente diseñar cada una de las piezas correctamente con respecto a los sistemas de coordenadas para que el ensamblaje sea más sencillo. También habrá que anotar las coordenadas de cada articulación con respecto al punto homólogo de la pieza siguiente para poder realizar el cálculo de las matrices de posicionamiento. Para ello se empleará el módulo de ensamblaje del programa. Se puede seguir el tutorial de creación y comprobación de ensamblajes en [Tutorial de comprobación de ensamblaje-20201218_105148.mkv](#)

Posteriormente se puede exportar cada fichero de pieza al formato STL para su carga en el programa de Python. Se debe prestar especial atención a que las dimensiones de las piezas están en **mm**, mientras que al exportarlas se tienen que pasar a **m**, unidades en las que está la simulación. Este proceso está descrito en [Conversión de SLDPRT a STL-20201218_042711.mkv](#)

5. Programa principal de control (Informática)

En la entrega de la práctica para la asignatura de Informática será necesario generar una animación en la que el brazo robótico parta de una posición de reposo, atrape una lata en la posición L2 y la lleve a la zona de descarga. Ser capaz de hacerlo también estando la lata en las posiciones L1 o L3 será tenido en cuenta con hasta un +2 en la nota de la práctica (siendo la nota máxima 10 en cualquier caso, y pudiendo obtenerse igualmente el 10 atrapando la lata solo en L2 si todo está **perfecto**).

El resultado final de la tarea se podrá visualizar mediante un programa escrito en el lenguaje Python. En este lenguaje hay una cantidad ingente de librerías que permiten realizar prácticamente cualquier tipo de tarea con relativa facilidad. En este caso se emplearán las siguientes:

- NumPy => Cálculo con arrays n-dimensionales, álgebra, procesado de señal...
- numpy-stl => Procesado de STL (carga y almacenamiento de ficheros, manipulación...)

- vtkplotlib => Representación simple de gráficos 3D basándose en VTK, NumPy y Matplotlib. Adicionalmente, en nuestro caso, usará también numpy-stl y PyQt6.

Como paso previo a la realización del programa que se entregará, será necesario tomar un par de decisiones:

1. Cuáles serán las posiciones clave de las piezas de la máquina y cuáles son las rotaciones y traslaciones necesarias para llegar a ellas. Por “posiciones clave” se entiende aquellas posiciones por las cuáles una pieza debe de pasar necesariamente para cumplir con el objetivo de la máquina (las posiciones intermedias entre las posiciones clave las deberá de calcular automáticamente el programa interpolando linealmente). Esto implicará “jugar” con las piezas para ver dónde quedan mejor.
2. Determinar en cuántos pasos de tiempo queremos dividir la animación. A mayor número de pasos, la animación será más suave pero más lenta, con lo que el valor óptimo dependerá de la potencia del ordenador. Se puede empezar con 100 pasos y ajustar cuando el programa esté finalizado.

El pseudocódigo de la aplicación, a grandes rasgos, es el siguiente:

1. Cargar los ficheros STL necesarios.
2. Crear la ventana de visualización.
3. Para cada posición clave:
 - a. Para cada paso de tiempo mientras la máquina no llegue a la posición final:
 - i. Hacer una copia de la máquina (para no tocar la original).
 - ii. Para cada pieza de la (copia de la) máquina:
 1. Determinar su rotación y traslación respecto al origen de coordenadas teniendo en cuenta que le aplicaremos también las de la pieza anterior si la hubiese.
 2. Calcular las matrices de rotación y traslación para esos valores.
 3. Calcular la matriz de transformación correspondiente.
 4. Calcular la matriz de transformación final como producto de la suya y la de la pieza anterior.
 5. Aplicar la matriz de transformación.
 - iii. Pintar las piezas en la ventana.

En clase, o en el propio Campus Online, se explicará esta práctica con el detalle que sea necesario / proveerá el material necesario para su comprensión, incluyendo el uso de numpy-stl y vtkplotlib.

Para instalar esos paquetes es necesario teclear lo siguiente en el terminal integrado de Visual Studio Code o en cualquier terminal de Windows (cmd o PowerShell):

```
pip install vtkplotlib numpy-stl PyQt6
```

A continuación, se muestran algunos ejemplos que pueden resultar útiles, los cuales fueron probados con las siguientes versiones de software el día 22/11/2025:

- Windows 11 Pro 25H2. En principio, cualquier versión de Windows 10 o Windows 11 debería de valer. Por supuesto, la práctica también se puede realizar con macOS o Linux. Sea como sea, si hay dudas al respecto de los requisitos técnicos, contactad a los profesores.
- Python 3.13.9. En principio, cualquier versión de Python 3.11, 3.12 o 3.13 debería de valer.
- Visual Studio Code 1.106.2

- vtkplotlib 2.1.1. Al instalar este paquete, se instalará automáticamente NumPy 2.3.5, VTK 9.5.2 y matplotlib 3.10.7, salvo que ya esté instalada alguna versión previa.
- numpy-stl 3.2.0
- PyQt6 6.10.0

Si alguien tiene un Mac con macOS Tahoe, tendrá que instalar la versión 6.9.1 de PyQt6:

```
pip install vtkplotlib numpy-stl PyQt6==6.9.1
```

5.1. Ejemplo de cómo mostrar un STL en pantalla (numpy-stl y vtkplotlib).

Este ejemplo es útil para comprobar que todo está bien instalado y asume que nuestro fichero se llama "modelo.stl", si se quiere usar otro, basta con cambiar el nombre.

```
import stl
import vtkplotlib as vpl

# Cargamos el modelo desde un fichero con nombre "modelo.stl".
modelo = stl.mesh.Mesh.from_file("modelo.stl")
# Crea la ventana (pero no la muestra).
vpl.QtFigure()
# Añadimos la malla del objeto a la ventana.
vpl.mesh_plot(modelo)
# Mostramos la ventana.
vpl.show()
```

5.2. Ejemplo de cómo crear una animación con ficheros STL (vtkplotlib).

Este ejemplo consiste en una función que podéis llamar desde vuestro programa para pintar cada paso de la animación. Es decir, para cada paso de tiempo, calcularéis las posiciones de las distintas piezas de la máquina y después llamaréis a esta función, pasándole una lista con las piezas, para que actualice lo que se ve en la ventana. Al hacer esto repetidamente se genera el efecto de la animación.

```
def paint(meshes):
    """
    Muestra los objetos en pantalla.

    meshes : lista con los objetos a visualizar.
    vpl_meshes : lista con los objetos que vamos pintando en pantalla y que se van a
        borrar después de pintarlos. Es necesaria porque vtkplotlib representa los
        objetos de una forma distinta a numpy-stl (guarda más información).
    """
    vpl_meshes = []
    for mesh in meshes:
        # Añadimos la malla del objeto a la ventana.
        vpl_mesh = vpl.mesh_plot(mesh)
        # Lo añadimos, en formato vpl, a otra lista para después poder borrarlo.
        vpl_meshes.append(vpl_mesh)
    # Obtenemos una referencia a la ventana actual.
    figure = vpl.gcf()
    # Recalculamos lo que se va a mostrar en la ventana.
```

```

figure.update()
# Lo mostramos, pero sin que el programa se detenga.
figure.show(block=False)
# Borramos de la ventana los objetos que acabamos de pintar, ya que la próxima vez que se
# ejecute esta función volverán a ser pintados, probablemente con sus coordenadas
# modificadas.
for mesh in vpl_meshes:
    figure.remove_plot(mesh)

```

5.3. Funciones de NumPy que pueden ser útiles.

Ejemplos utilizando algunas funciones de NumPy que quizás os resulten útiles para la realización de la práctica.

```

import numpy

# Asigna el número 'pi' a la variable 'x'.
x = numpy.pi
# Pasa 60º a radianes.
angulo = numpy.radians(60)
# Calcula el coseno de 60º y lo guarda en 'coseno'.
coseno = numpy.cos(angulo)
# Calcula el seno de 60º y lo guarda en 'seno'.
seno = numpy.sin(angulo)
# Crea una matriz identidad de tamaño 4x4 y la guarda en 'm'.
Rz = numpy.identity(4)
# Junto con la sentencia anterior, crea una matriz de transformación
# en la que solo rota sobre el eje z, sin traslación alguna tampoco.
# https://es.wikipedia.org/wiki/Matriz_de_rotación#Rotaciones_básicas
Rz[0:3, 0:3] = [
    [numpy.cos(angulo), -numpy.sin(angulo), 0.0],
    [numpy.sin(angulo), numpy.cos(angulo), 0.0],
    [0.0, 0.0, 1.0],
]
# Suponiendo que se hayan calculado en otro sitio las otras matrices de
# rotación y la de traslación, la matriz de transformación se puede
# calcular así (recuerda que el operador @ multiplica matrices de NumPy):
Mt = T @ Rz @ Ry @ Rx

```

5.4. Ejemplo final.

Ejemplo utilizando algunas de las funciones mencionadas anteriormente que carga un modelo, lo rota 90º sobre el eje Z y lo muestra por pantalla. Obviamente, la multiplicación por las matrices identidad es innecesaria, solo se muestra para facilitar la comprensión.

```

import numpy
import stl
import vtkplotlib as vpl

def rotate_z(angle):
    """Create a transformation matrix for rotation over z axis."""
    matrix = numpy.identity(4)

```

```

matrix[0:3, 0:3] = [
    [numpy.cos(angle), -numpy.sin(angle), 0.0],
    [numpy.sin(angle), numpy.cos(angle), 0.0],
    [0.0, 0.0, 1.0],
]
return matrix

def main():
    # Crea la ventana
    vpl.QtFigure()
    # Carga el modelo 'pieza.stl' y lo guarda en 'pieza'.
    pieza = stl.mesh.Mesh.from_file("pieza.stl")
    # Rota 'pieza' 90º sobre el eje 'z'.
    T = numpy.identity(4)
    Rz = rotate_z(numpy.radians(90.0))
    Ry = numpy.identity(4)
    Rx = numpy.identity(4)
    Mt = T @ Rz @ Ry @ Rx
    pieza.transform(Mt)
    # Muestra la pieza
    vpl.mesh_plot(pieza)
    vpl.show()

if __name__ == "__main__":
    main()

```

5.5. Entrega

*En este curso 2025/26, la entrega de esta práctica será **opcional**. Para aquellos estudiantes que la realicen, su valor será de un 15% en la nota total de la asignatura (un 25% de la parte práctica), tal y como se explica en la guía docente. Los que decidan no entregarla, sumarán ese 15% a la parte práctica del examen final. En cualquier caso, la norma de que la nota de la primera prueba de programación + la nota de la práctica deben sumar 5 puntos queda anulada.*

La práctica debe realizarse de forma individual.

El plazo de entrega finaliza el domingo 14 de diciembre de 2025 a las 23:55 horas (nota: no se resolverán dudas los fines de semana).

Es necesario enviar antes de ese momento el o los fichero(s) .py que contiene(n) el código fuente de la práctica a través del Campus Online de la UDC (<https://udconline.udc.gal/>), en la sección *Entrega de la práctica*, utilizando el enlace que allí aparece.

Dicho fichero deberá ser renombrado antes de subirlo a la página web de la asignatura de forma que su nombre sea *apellido1_apellido2_nombre.py* donde *apellido1*, *apellido2* y *nombre* son los apellidos y el nombre del alumno que entrega la práctica.

Las notas se publicarán en el Campus Online como muy tarde el 30 de diciembre de 2025.

Se realizará una **defensa de la práctica entre los días 15 y 19 de diciembre de 2025**. Para ello, se habilitará en la página web de la asignatura un procedimiento para concertar una cita con los profesores de la asignatura. La cita debe concertarla cada alumno con su profesor de prácticas correspondiente. **El resultado de esta defensa podrá bajar la nota de la práctica.**

La defensa consistirá en responder al profesor una serie de preguntas al respecto de la práctica. **¡Una práctica perfecta puede ocasionar un 0 si las respuestas no son adecuadas! Si no se realiza la defensa la práctica no puntuará en la nota final.**

Si se detectan varias prácticas iguales o muy similares todos los alumnos implicados tendrán un 0 como nota, incluido el autor original si estamos ante una práctica realizada por un alumno y copiada por otros. A este efecto se usará un detector de plagios y **todas las prácticas con un porcentaje de parecido superior al 50% se considerarán suspensas con un 0.**

5.6. Evaluación

Para puntuar la práctica se valorará lo siguiente:

- Que el programa funcione correctamente.
- La eficiencia en la implementación.
- La organización **correcta** del programa en funciones (30% de la nota).
- El uso **adecuado** de listas (15% de la nota) y diccionarios (15% de la nota). Por ejemplo, se podría usar una lista para las piezas, donde cada pieza fuese un diccionario con campos para la malla, el nombre de la pieza y el nombre de la pieza a la que está conectada.
- El uso de técnicas adecuadas de programación tales como: **no** utilización de variables globales, uso de nombres significativos para las variables y las funciones, existencia de comentarios explicativos, etc.
- La adecuación y rapidez en las respuestas a las preguntas del profesor en la defensa de la práctica por parte del alumno.